

```

import os
import logging
from telegram import Update, ReplyKeyboardMarkup, KeyboardButton
from telegram.ext import Application, CommandHandler, MessageHandler, filters, Co

# Bot token
TOKEN = os.environ.get("BOT_TOKEN", "8905229771:AAEbQrJ6KVTz87hspuY7ya2N4HNtnBkR6")

# Logging
logging.basicConfig(format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# Conversation states
CHOOSING_SERVICE, CHOOSING_ZONE, CHOOSING_WEIGHT, CHOOSING_BUSSTOP, CHOOSING_ERRA

# Pricing
ZONE_PRICES = {
    "Zone 1 - On Campus": {"Light": 300, "Medium": 500, "Heavy": 700},
    "Zone 2 - Near Off Campus": {"Light": 500, "Medium": 700, "Heavy": 900},
    "Zone 3 - Mid Off Campus": {"Light": 700, "Medium": 900, "Heavy": 1100},
    "Zone 4 - Far Off Campus": {"Light": 1200, "Medium": 1400, "Heavy": 1600},
}

ERRAND_FEES = {
    "Simple Errand / Food Order": 100,
    "Complex Errand / Bulk Shopping": 250,
}

EXPRESS_SURCHARGE = 300
DISTANCE_MODIFIER = 200

ZONE_LOCATIONS = {
    "Zone 1 - On Campus": "Anywhere within FUNAAB campus",
    "Zone 2 - Near Off Campus": "Harmony, Accord, Zoo, Oluwo, Isolu",
    "Zone 3 - Mid Off Campus": "Iabuta, Camp (from Accord/Zoo/Oluwo/Isolu)",
    "Zone 4 - Far Off Campus": "Camp (from Harmony), Town",
}

# Helper: main menu keyboard
def main_menu():
    keyboard = [
        [KeyboardButton("📦 Send a Package"), KeyboardButton("🛒 Errand / Food /
        [KeyboardButton("⚡ Express Delivery"), KeyboardButton("📅 Schedule Deliv
        [KeyboardButton("💰 Price Quote"), KeyboardButton("🗺️ View Zones")],

```

```

        [KeyboardButton("💳 Payment Info"), KeyboardButton("☎ Contact Us")],
        [KeyboardButton("ℹ About BikeBlitz")],
    ]
    return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

def zone_keyboard():
    keyboard = [
        [KeyboardButton("Zone 1 - On Campus")],
        [KeyboardButton("Zone 2 - Near Off Campus")],
        [KeyboardButton("Zone 3 - Mid Off Campus")],
        [KeyboardButton("Zone 4 - Far Off Campus")],
        [KeyboardButton("🏠 Main Menu")],
    ]
    return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

def weight_keyboard():
    keyboard = [
        [KeyboardButton("Light (fits in one hand)")],
        [KeyboardButton("Medium (requires two hands)")],
        [KeyboardButton("Heavy (requires effort to carry)")],
        [KeyboardButton("Very Heavy (10kg+)")],
        [KeyboardButton("🏠 Main Menu")],
    ]
    return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

def busstop_keyboard():
    keyboard = [
        [KeyboardButton("✅ Close to bus stop")],
        [KeyboardButton("⚠ Far from bus stop (+N200)")],
        [KeyboardButton("🏠 Main Menu")],
    ]
    return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

def errand_keyboard():
    keyboard = [
        [KeyboardButton("Simple Errand / Food Order")],
        [KeyboardButton("Complex Errand / Bulk Shopping")],
        [KeyboardButton("🏠 Main Menu")],
    ]
    return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

def delivery_type_keyboard():
    keyboard = [
        [KeyboardButton("⚡ Express Delivery")],
        [KeyboardButton("📅 Schedule Delivery")],
        [KeyboardButton("🏠 Main Menu")],
    ]

```

```

        return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

def confirm_keyboard():
    keyboard = [
        [KeyboardButton("✅ Confirm Order")],
        [KeyboardButton("❌ Cancel Order")],
        [KeyboardButton("🏠 Main Menu")],
    ]
    return ReplyKeyboardMarkup(keyboard, resize_keyboard=True)

# Map weight button text to weight category
def parse_weight(text):
    if "Light" in text:
        return "Light"
    elif "Medium" in text:
        return "Medium"
    elif "Heavy (requires effort)" in text:
        return "Heavy"
    elif "Very Heavy" in text:
        return "Very Heavy"
    return None

# START
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    await update.message.reply_text(
        "👋 Welcome to *BikeBlitz* 🚲\n\n"
        "FUNAAB's fastest campus delivery and errand service.\n\n"
        "🕒 *Operating Hours:* Daily 9am - 9pm\n\n"
        "📦 *Same-day cut-off:* 8pm\n\n"
        "Fast. Reliable. Zero silence. Every order.\n\n"
        "What would you like to do today? 📌",
        parse_mode="Markdown",
        reply_markup=main_menu()
    )
    return CHOOSING_SERVICE

# ABOUT
async def about(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "📄 *About BikeBlitz*\n\n"
        "BikeBlitz is a student-powered campus delivery and errand service based"
        "We handle:\n"
        "📦 Package and document delivery\n"
        "🛒 Errands and market runs\n"
        "🍔 Food pickup and delivery\n\n"
        "Our riders are FUNAAB students who know the campus inside out.\n\n"
    )

```

```

        "Fast. Reliable. Community-driven. 🚲",
        parse_mode="Markdown",
        reply_markup=main_menu()
    )

    return CHOOSING_SERVICE

# ZONES
async def zones(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = (
        "🚲 *BikeBlitz Delivery Zones*\n\n"
        "📍 *Zone 1 – On Campus*\n"
        "Anywhere within FUNAAB campus\n\n"
        "📍 *Zone 2 – Near Off Campus*\n"
        "Harmony, Accord, Zoo, Oluwo, Isolu\n\n"
        "📍 *Zone 3 – Mid Off Campus*\n"
        "Labuta, Camp (from Accord/Zoo/Oluwo/Isolu)\n\n"
        "📍 *Zone 4 – Far Off Campus*\n"
        "Camp (from Harmony), Town and beyond\n\n"
        "_Note: A ₦200 distance modifier applies if your location is far from the
    )

    await update.message.reply_text(text, parse_mode="Markdown", reply_markup=mai
    return CHOOSING_SERVICE

# PRICING
async def pricing(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = (
        "💰 *BikeBlitz Price List*\n\n"
        "—————\n"
        "*B2B – Package Delivery*\n"
        "—————\n"
        "Zone 1: Light ₦300 | Medium ₦500 | Heavy ₦700\n"
        "Zone 2: Light ₦500 | Medium ₦700 | Heavy ₦900\n"
        "Zone 3: Light ₦700 | Medium ₦900 | Heavy ₦1,100\n"
        "Zone 4: Light ₦1,200 | Medium ₦1,400 | Heavy ₦1,600\n\n"
        "🛵 Far from bus stop: +₦200\n"
        "⚡ Express delivery: +₦300\n\n"
        "—————\n"
        "*B2C – Errands & Food*\n"
        "—————\n"
        "Zone fee + Service fee = Total charge\n\n"
        "Simple errand/food: +₦100\n"
        "Complex errand/bulk shopping: +₦250\n\n"
        "_Item cost is paid directly to vendor._\n\n"
        "⚖️ *Weight Guide:*\n"
        "Light = fits in one hand\n"
        "Medium = requires two hands\n"
        "Heavy = requires effort to carry\n"
    )

```

```

        "Very Heavy = 10kg+ (negotiated)"
    )
    await update.message.reply_text(text, parse_mode="Markdown", reply_markup=mai
    return CHOOSING_SERVICE

# PAYMENT
async def payment(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "\ud83d\udcbb *Payment Information*\n\n"
        "BikeBlitz requires payment of the delivery charge *before* your rider mo
        "\ud83d\udcbb *How it works:*\n"
        "\ud83d\udcbb 1 Place your order\n"
        "\ud83d\udcbb 2 Receive your price quote\n"
        "\ud83d\udcbb 3 Transfer the delivery charge to our account\n"
        "\ud83d\udcbb 4 Send your receipt screenshot to confirm\n"
        "\ud83d\udcbb 5 Your rider moves immediately ⚡\n\n"
        "\ud83d\udcbb *Bank Details:*\n"
        "Bank: [Your Bank Name]\n"
        "Account Number: [Your Account Number]\n"
        "Account Name: BikeBlitz\n\n"
        "_For B2C orders: item cost is paid directly by you to the vendor when yo
        parse_mode="Markdown",
        reply_markup=main_menu()
    )
    return CHOOSING_SERVICE

# CONTACT
async def contact(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "\ud83d\udcde *Contact BikeBlitz*\n\n"
        "Need to speak to a team member directly?\n\n"
        "\ud83d\udcde WhatsApp: [Your WhatsApp Number]\n"
        "\ud83d\udcde Email: [Your Email]\n"
        "\ud83d\udcde Location: FUNAAB Campus, Abeokuta\n\n"
        "\ud83d\udcde Available daily 9am - 9pm\n\n"
        "_For orders and quotes, use the menu below for faster service._",
        parse_mode="Markdown",
        reply_markup=main_menu()
    )
    return CHOOSING_SERVICE

# B2B FLOW
async def b2b_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["service"] = "B2B"
    context.user_data["delivery_type"] = "Standard"
    await update.message.reply_text(
        "\ud83d\udcbb *Package Delivery*\n\n"

```

```

        "Let's get your package delivered!\n\n"
        "First, select your delivery zone 📌\n\n"
        "Not sure which zone? Use /zones to check.",
        parse_mode="Markdown",
        reply_markup=zone_keyboard()
    )
    return CHOOSING_ZONE

# B2C FLOW
async def b2c_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["service"] = "B2C"
    context.user_data["delivery_type"] = "Standard"
    await update.message.reply_text(
        "🛒 *Errand / Food / Market Run*\n\n"
        "We'll go buy or collect it for you!\n\n"
        "First, what type of errand is this? 📌",
        parse_mode="Markdown",
        reply_markup=errand_keyboard()
    )
    return CHOOSING_ERRAND

# EXPRESS
async def express_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["delivery_type"] = "Express"
    await update.message.reply_text(
        "⚡ *Express Delivery*\n\n"
        "Need it done urgently? We've got you!\n\n"
        "Express delivery adds *$300* on top of your zone price for priority hand
        "Is this a package delivery or an errand? 📌",
        parse_mode="Markdown",
        reply_markup=ReplyKeyboardMarkup([
            [KeyboardButton("📦 Package Delivery")],
            [KeyboardButton("🛒 Errand / Food / Market")],
            [KeyboardButton("🏠 Main Menu")],
        ], resize_keyboard=True)
    )
    return CHOOSING_SERVICE

# SCHEDULE
async def schedule_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["delivery_type"] = "Scheduled"
    await update.message.reply_text(
        "📅 *Schedule a Delivery*\n\n"
        "Plan ahead and we'll handle it at your preferred time!\n\n"
        "📌 *Rules:*\n\n"
        "• Minimum 1 hour notice required\n\n"
        "• Same-day scheduling closes at 8pm\n\n"

```

```

        "• Available daily 9am - 9pm\n\n"
        "Please type your preferred date and time:\n"
        "(Example: Tomorrow 2pm or Monday 10am)",
        parse_mode="Markdown",
        reply_markup=ReplyKeyboardMarkup([[KeyboardButton("🏠 Main Menu")]], resi
    )
    return SCHEDULING_TIME

async def handle_schedule_time(update: Update, context: ContextTypes.DEFAULT_TYPE):
    scheduled_time = update.message.text
    context.user_data["scheduled_time"] = scheduled_time
    await update.message.reply_text(
        f"✅ Got it! Scheduled for: {scheduled_time}\n\n"
        "Now, is this a package delivery or an errand? 📌",
        parse_mode="Markdown",
        reply_markup=ReplyKeyboardMarkup([
            [KeyboardButton("📦 Package Delivery")],
            [KeyboardButton("🛒 Errand / Food / Market")],
            [KeyboardButton("🏠 Main Menu")],
        ], resize_keyboard=True)
    )
    return CHOOSING_SERVICE

# HANDLE ZONE SELECTION
async def handle_zone(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "🏠 Main Menu":
        return await start(update, context)

    if text not in ZONE_PRICES:
        await update.message.reply_text("Please select a valid zone 📌", reply_ma
        return CHOOSING_ZONE

    context.user_data["zone"] = text
    service = context.user_data.get("service", "B2B")

    if service == "B2B":
        await update.message.reply_text(
            f"✅ Zone selected: {text}\n\n"
            "Now, how heavy is your package? 📌",
            parse_mode="Markdown",
            reply_markup=weight_keyboard()
        )
        return CHOOSING_WEIGHT
    else:
        await update.message.reply_text(
            f"✅ Zone selected: {text}\n\n"

```

```

        "Is your pickup/dropoff location close to the main bus stop or far fr
        parse_mode="Markdown",
        reply_markup=busstop_keyboard()
    )
    return CHOOSING_BUSSTOP

# HANDLE ERRAND TYPE
async def handle_errand(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "🏠 Main Menu":
        return await start(update, context)

    if text not in ERRAND_FEES:
        await update.message.reply_text("Please select a valid errand type 📌", r
        return CHOOSING_ERRAND

    context.user_data["errand_type"] = text
    context.user_data["errand_fee"] = ERRAND_FEES[text]

    await update.message.reply_text(
        f"✅ Errand type: {text}\n\n"
        "Now select your delivery zone 📌",
        parse_mode="Markdown",
        reply_markup=zone_keyboard()
    )
    return CHOOSING_ZONE

# HANDLE WEIGHT
async def handle_weight(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "🏠 Main Menu":
        return await start(update, context)

    weight = parse_weight(text)
    if not weight:
        await update.message.reply_text("Please select a valid weight 📌", reply_
        return CHOOSING_WEIGHT

    if weight == "Very Heavy":
        await update.message.reply_text(
            "⚖️ *Very Heavy Package*\n\n"
            "Packages above 10kg need to be negotiated separately.\n\n"
            "Please contact us directly:\n"
            "📱 WhatsApp: [Your Number]\n\n"
            "We'll get back to you immediately! ⚡",
            parse_mode="Markdown",
            reply_markup=main_menu()

```

```

    )
    return CHOOSING_SERVICE

context.user_data["weight"] = weight

await update.message.reply_text(
    f"✅ Weight: *{weight}*\n\n"
    "Is your dropoff location close to the main bus stop or far from it? 📌",
    parse_mode="Markdown",
    reply_markup=busstop_keyboard()
)
return CHOOSING_BUSSTOP

# HANDLE BUS STOP
async def handle_busstop(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "🏠 Main Menu":
        return await start(update, context)

    far_from_busstop = "Far" in text
    context.user_data["far_from_busstop"] = far_from_busstop

    # Calculate price
    zone = context.user_data.get("zone")
    service = context.user_data.get("service", "B2B")
    delivery_type = context.user_data.get("delivery_type", "Standard")

    if service == "B2B":
        weight = context.user_data.get("weight")
        base_price = ZONE_PRICES[zone][weight]
        distance_add = DISTANCE_MODIFIER if far_from_busstop else 0
        express_add = EXPRESS_SURCHARGE if delivery_type == "Express" else 0
        total = base_price + distance_add + express_add

        context.user_data["total"] = total
        context.user_data["base_price"] = base_price
        context.user_data["distance_add"] = distance_add
        context.user_data["express_add"] = express_add

    breakdown = (
        f"📄 *Order Summary*\n\n"
        f"📦 Service: Package Delivery\n"
        f"🌍 Zone: {zone}\n"
        f"⚖️ Weight: {weight}\n"
        f"🚚 Delivery Type: {delivery_type}\n"
        f"📍 Far from bus stop: {'Yes' if far_from_busstop else 'No'}\n\n"
        f"—————\n"
    )

```

```

        f"💰 Base price: ₦{base_price:,}\n"
    )
    if distance_add:
        breakdown += f"🛣️ Distance modifier: +₦{distance_add:,}\n"
    if express_add:
        breakdown += f"⚡ Express surcharge: +₦{express_add:,}\n"
    breakdown += (
        f"—————\n"
        f"🛒 *Total Delivery Charge: ₦{total:,}*~\n\n"
        f"Rider earns: ₦{int(total * 0.7):,} (70%)\n"
        f"BikeBlitz: ₦{int(total * 0.3):,} (30%)\n\n"
        f"Would you like to confirm this order? 📌"
    )

else:
    errand_fee = context.user_data.get("errand_fee", 100)
    errand_type = context.user_data.get("errand_type")
    base_price = ZONE_PRICES[zone]["Light"]
    distance_add = DISTANCE_MODIFIER if far_from_busstop else 0
    express_add = EXPRESS_SURCHARGE if delivery_type == "Express" else 0
    total = base_price + errand_fee + distance_add + express_add

    context.user_data["total"] = total
    context.user_data["base_price"] = base_price
    context.user_data["distance_add"] = distance_add
    context.user_data["express_add"] = express_add

    breakdown = (
        f"📋 *Order Summary*\n\n"
        f"🛒 Service: {errand_type}\n"
        f"🗺️ Zone: {zone}\n"
        f"🚲 Delivery Type: {delivery_type}\n"
        f"📍 Far from bus stop: {'Yes' if far_from_busstop else 'No'}\n\n"
        f"—————\n"
        f"💰 Zone delivery fee: ₦{base_price:,}\n"
        f"⚙️ Service fee: ₦{errand_fee:,}\n"
    )
    if distance_add:
        breakdown += f"🛣️ Distance modifier: +₦{distance_add:,}\n"
    if express_add:
        breakdown += f"⚡ Express surcharge: +₦{express_add:,}\n"
    breakdown += (
        f"—————\n"
        f"🛒 *Total Delivery Charge: ₦{total:,}*~\n\n"
        f"_Item cost paid directly to vendor_\n\n"
        f"Would you like to confirm this order? 📌"
    )

```

```

if delivery_type == "Scheduled":
    scheduled_time = context.user_data.get("scheduled_time", "")
    breakdown += f"\n📅 Scheduled for: *{scheduled_time}*"

await update.message.reply_text(breakdown, parse_mode="Markdown", reply_marku
return CONFIRMING_ORDER

# CONFIRM ORDER
async def handle_confirm(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "🏠 Main Menu" or text == "❌ Cancel Order":
        await update.message.reply_text(
            "❌ Order cancelled.\n\nNo worries – start a new order anytime! 🚲",
            reply_markup=main_menu()
        )
        context.user_data.clear()
        return CHOOSING_SERVICE

    if text == "✅ Confirm Order":
        total = context.user_data.get("total", 0)
        delivery_type = context.user_data.get("delivery_type", "Standard")
        scheduled_time = context.user_data.get("scheduled_time", "")

        confirmation = (
            f"✅ *Order Confirmed!*\n\n"
            f"💰 *Total Delivery Charge: ₦{total:,}*\n\n"
            f"Please transfer ₦{total:,} to:\n"
            f"🏦 Bank: [Your Bank Name]\n"
            f"📄 Account: [Your Account Number]\n"
            f"👤 Name: BikeBlitz\n\n"
            f"After payment:\n"
            f"1️⃣ Send your receipt screenshot here\n"
            f"2️⃣ Your rider will be dispatched immediately ⚡\n\n"
        )

    if delivery_type == "Scheduled":
        confirmation += f"📅 Your delivery is scheduled for: *{scheduled_time}*"

    confirmation += (
        f"📞 Questions? Contact us:\n"
        f"WhatsApp: [Your Number]\n\n"
        f"Thank you for choosing BikeBlitz! 🚲"
    )

    await update.message.reply_text(confirmation, parse_mode="Markdown", repl
    context.user_data.clear()

```

```

        return CHOOSING_SERVICE

# HANDLE ALL TEXT MESSAGES
async def handle_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text

    if text == "🏠 Main Menu":
        return await start(update, context)
    elif text == "📦 Send a Package" or text == "📦 Package Delivery":
        context.user_data["service"] = "B2B"
        if not context.user_data.get("delivery_type"):
            context.user_data["delivery_type"] = "Standard"
        await update.message.reply_text(
            "📦 *Package Delivery*\n\nSelect your delivery zone 📌",
            parse_mode="Markdown",
            reply_markup=zone_keyboard()
        )
        return CHOOSING_ZONE
    elif text == "🛒 Errand / Food / Market" or text == "🛒 Errand / Food / Market":
        context.user_data["service"] = "B2C"
        if not context.user_data.get("delivery_type"):
            context.user_data["delivery_type"] = "Standard"
        await update.message.reply_text(
            "🛒 *Errand / Food / Market Run*\n\nWhat type of errand? 📌",
            parse_mode="Markdown",
            reply_markup=errand_keyboard()
        )
        return CHOOSING_ERRAND
    elif text == "⚡ Express Delivery":
        return await express_start(update, context)
    elif text == "📅 Schedule Delivery":
        return await schedule_start(update, context)
    elif text == "💰 Price Quote":
        return await pricing(update, context)
    elif text == "🗺️ View Zones":
        return await zones(update, context)
    elif text == "💳 Payment Info":
        return await payment(update, context)
    elif text == "📞 Contact Us":
        return await contact(update, context)
    elif text == "ℹ️ About BikeBlitz":
        return await about(update, context)
    else:
        await update.message.reply_text(
            "I didn't understand that. Please use the menu below 📌",
            reply_markup=main_menu()
        )

```

```

        return CHOOSING_SERVICE

def main():
    app = Application.builder().token(TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[
            CommandHandler("start", start),
            MessageHandler(filters.TEXT & ~filters.COMMAND, handle_text),
        ],
        states={
            CHOOSING_SERVICE: [MessageHandler(filters.TEXT & ~filters.COMMAND, ha
            CHOOSING_ZONE: [MessageHandler(filters.TEXT & ~filters.COMMAND, handl
            CHOOSING_WEIGHT: [MessageHandler(filters.TEXT & ~filters.COMMAND, han
            CHOOSING_BUSSTOP: [MessageHandler(filters.TEXT & ~filters.COMMAND, ha
            CHOOSING_ERRAND: [MessageHandler(filters.TEXT & ~filters.COMMAND, han
            CONFIRMING_ORDER: [MessageHandler(filters.TEXT & ~filters.COMMAND, ha
            SCHEDULING_TIME: [MessageHandler(filters.TEXT & ~filters.COMMAND, han
        },
        fallbacks=[CommandHandler("start", start)],
    )

    app.add_handler(conv_handler)
    app.add_handler(CommandHandler("zones", zones))
    app.add_handler(CommandHandler("pricing", pricing))
    app.add_handler(CommandHandler("payment", payment))
    app.add_handler(CommandHandler("contact", contact))
    app.add_handler(CommandHandler("about", about))

    print("BikeBlitz Bot is running...")
    app.run_polling()

if __name__ == "__main__":
    main()

```